

Patient Tracker System

Sarah Sweet

Luddy School of Informatics, Computing, and Engineering, Indiana University Indianapolis

INFO-C 450: System Design

Jafrina Jabin

December 16, 2025

Table of Contents

- Cover Page: Page 1
- Table of Contents: Page 2
- Customer Problem Statements and System Requirements: Page 3-10
- Functional Requirement Specification: Page 11-13
- System Sequence Diagram: Page 14-18
- Activity Diagram: Page 17-18
- Traceability Matrix: Page 19-21
- System Architecture and System Design: Page 22-24
- Algorithms and Data Structures: Page 25-26
- User Interface Design and Implementation, Design of Tests: Page 27-32
- Project Plan: Page 33-34

Customer Problem Statements & System Requirements

Problem Statement:

A visit to the doctor's office or hospital is often met with stress from the start. It does not help that upon arrival, the patient is expected to fill out paperwork that feels excessive. There are many forms that patients fill out each time they enter a healthcare setting. There are still hospitals that use paper records instead of electronic file keeping, which can slow down the process of receiving results and make it more difficult for patients to access their health information.

A new patient tracker system should be electronic and keep all patient information and records in one easily accessible place. Patients should be able to log into the system to set up appointments, see test results, and access other relevant health information. This would speed up getting answers about health concerns and allow patients to stay involved in their own health. Patients would not have to wait for calls from providers and administration, the information would all be available to them immediately.

Glossary of Terms:

- Appointment Management- Schedule, reschedule, or cancel appointments
- Reminders- Notifications sent straight to a patient's phone or email address, reminding them to confirm appointments.
- Medical Record- A patient's medical history with past and current health issues. Test results and patient medications should also be listed here.
- Patient tracker- A portal for patients to see all their health information in one place.

System Requirements: •

Functional requirements:

No.	Priority Weight	Description
REQ-1	High	Patient creates an account in the portal. Here they will be able to view all their medical information.
REQ-2	High	Patients can view, schedule, and cancel appointments to see providers from their portal account.
REQ-3	High	Patients can view their test results as soon as they are available.
REQ-4	High	Doctors and other medical staff can update records, so patients can see up to date treatment plans.
REQ-5	High	Patients can sign up to receive reminders of upcoming appointments and can
		reschedule or cancel if needed.

REQ-6	Medium	Patients can communicate directly with their providers and request prescription refills straight from the portal.
REQ-7	Medium	Administrative workers should be able to go into the system and make appointment changes.
REQ-8	Medium	Doctors and other medical professionals can use the portal to update availability and notify patients when they need to reschedule or cancel appointments.

- Nonfunctional Requirements:

No.	Priority Weight	Description
NFR-1	High	Functionality- The patient tracker system must allow patients, medical staff,
		administration, etc. to access and use the portal.

NFR-2	High	Usability- The patient tracker system should be selfexplanatory and easily accessible for all users regardless of age, disabilities, etc.
NFR-3	High	Reliability- The patient tracker must be able to support many users at one time. The information stored on the system should be secure.
NFR-4	High	Performance- Users should be able to access necessary information quickly.
NFR-5	Medium	Supportability- The system should regularly be updated and ensure that the system does not go down during popular times/business hours.

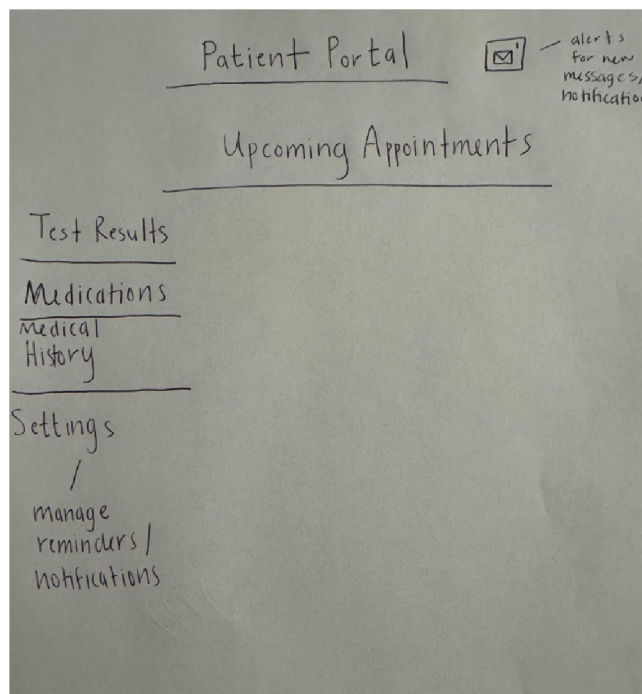
- User Interface Requirements:

No.	Priority	Description
-----	----------	-------------

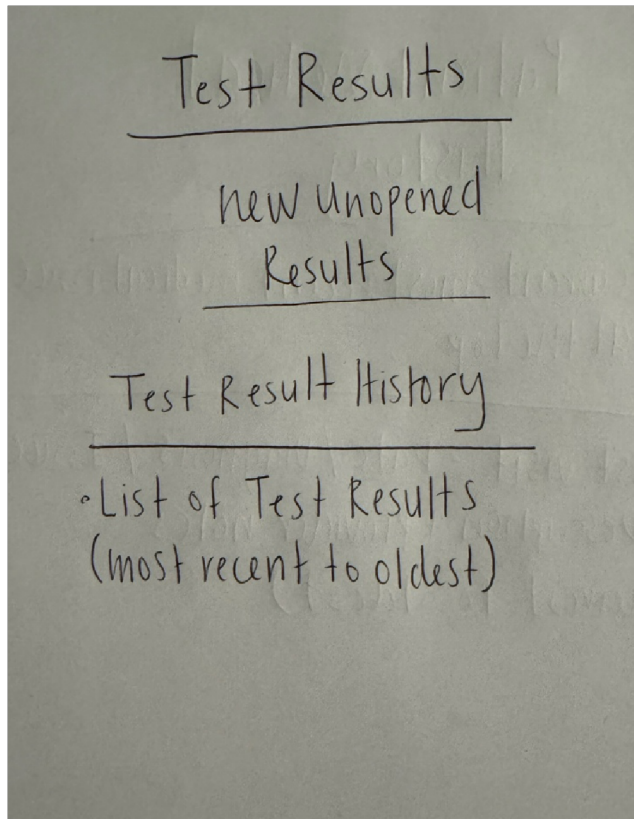
UI REQ-1	High	Patient portal home screen that shows appointments and alerts for new test results, messages from providers, updates to prescriptions, etc. Different tabs displayed help patients access full medical information, test results, etc.
UI REQ-2	High	Patients can view test results. New test results that have not been viewed yet will be highlighted in some way.
UI REQ-3	High	Patients can view full medical records. Here they will find current treatment plans and past medical information. Their full health history is completely accessible starting with the newest information at the top.
UI REQ-4	High	Administrators and medical staff (doctors, nurses, etc.)

		can see a staff view of the portal. There is a calendar to view all upcoming appointments. They are also able to access a list of current patients and navigate to these users. Here they can update records, send new prescriptions and refills, and communicate with patients if needed.
--	--	---

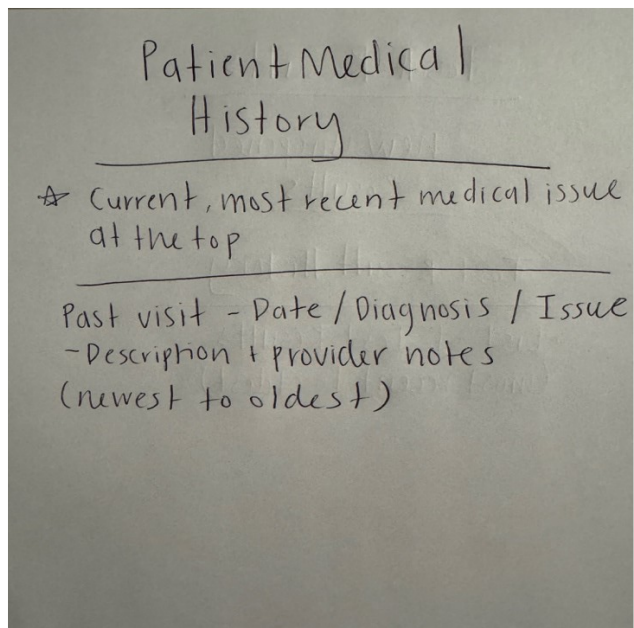
Patient Portal Home Screen Sketch



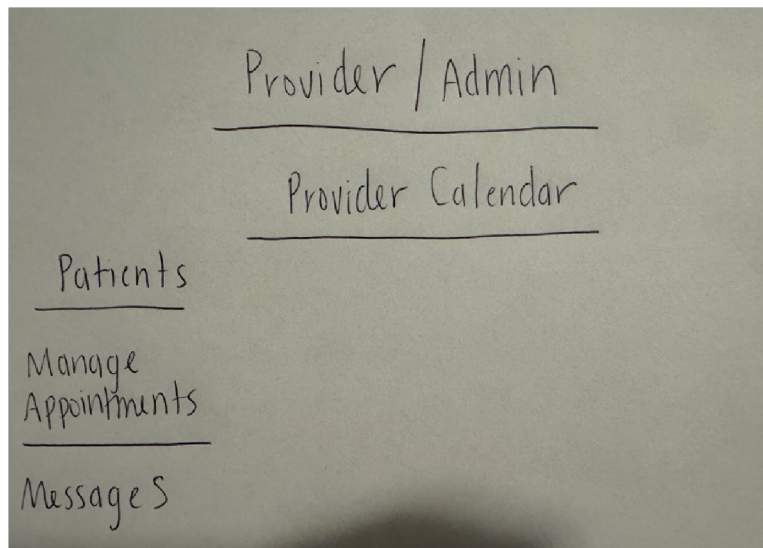
Test Result Page Sketch



Medical History Page Sketch



Provider/Admin Screen Sketch



Plan of Work:

Completed:

- Proposal
- Glossary of Terms
- System Requirements
- Sketched ideas of what the different screens in the portal will look like.

I have not started the actual coding yet, but I will begin on that this week. The work that I included in my timeline in my proposal will start to get checked off soon.

Functional Requirements Specification of Patient Tracker System

Stakeholders

- Patients
- Providers and Staff
- Admission Workers

Actors and Goals

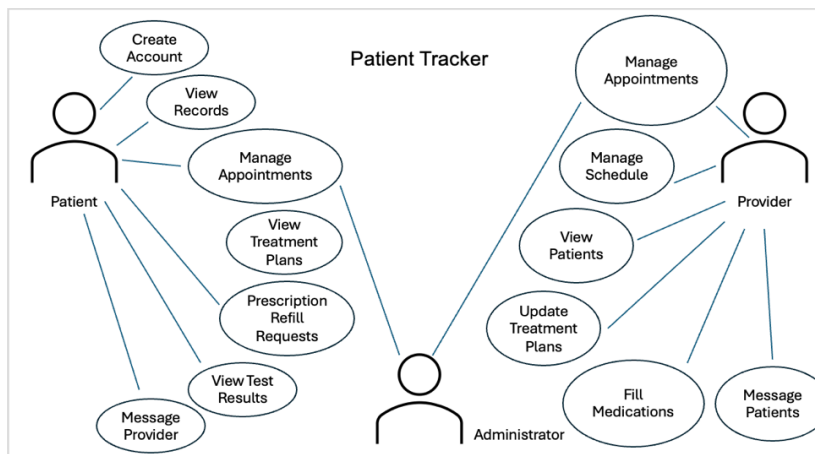
- Primary Actors
 - Patients can access all their medical records via the patient tracker system. They can set up an account without assistance or work. They can schedule appointments and request prescription refills from their doctors. Patients should be able to view their test results and even chat with their provider directly within the system.
 - Doctors and Staff can view their schedules and make changes as needed. They can see all their patients in the system. They can add to patients' treatment plans, fill prescriptions, and answer questions the patient asks.
- Secondary Actors
 - Administration Workers can go into the patient tracker system and manage appointments and add to the doctors' schedules. They can basically do everything for the patient via the system, but getting patients involved in their healthcare by giving them full access is the point of the patient tracker system.

Use Cases

- Patient can create an account on their own (2).

- Patients can view full medical records (3).
- Patients can view their current treatment plans (2).
- Patients can schedule, reschedule, and cancel appointments in the system (3).
- Patients can request prescription refills (3).
- Patients can view their test results (3).
- Patients can talk with their providers (4).
- Providers can view their schedules (2).
- Providers can view patients (2).
- Providers can add to the patient treatment plans (3).
- Providers can fill prescriptions (3).
- Providers can communicate with their patients (4).
- Administrators can manage appointments (3).
- Administrators can schedule new appointments and add them to the provider schedules (3).
- Total: 40, which is about 20 engineer days

Case Diagram



System Sequence Diagram and Activity Diagram

Patient Tracker System

System Sequence Diagrams

I chose “Schedule an Appointment” and “Request a Refill for a Prescription” to show sequence diagram examples for the Patient Tracker System.

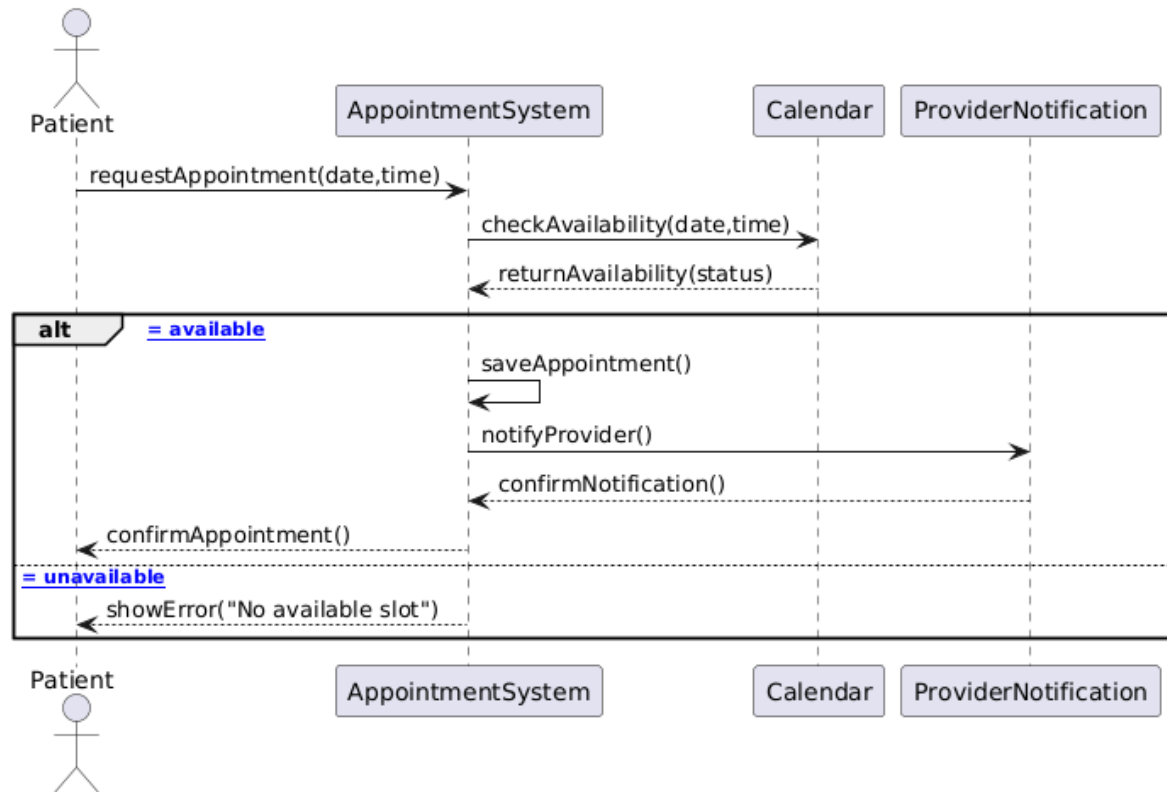
Schedule an Appointment:

Actor: Patient

Object: AppointmentSystem, Calendar, ProviderNotification

Steps of this interaction:

1. The patient requests a date and time for their appointment.
2. The patient tracker system checks the provider’s availability in the Calendar.
3. The calendar returns whether that appointment is available.
4. The system approves the appointment and stores it.
5. The system then notifies the provider of the appointment.
6. The patient gets confirmation from the system when the provider accepts the appointment.



Request a Refill for a Prescription:

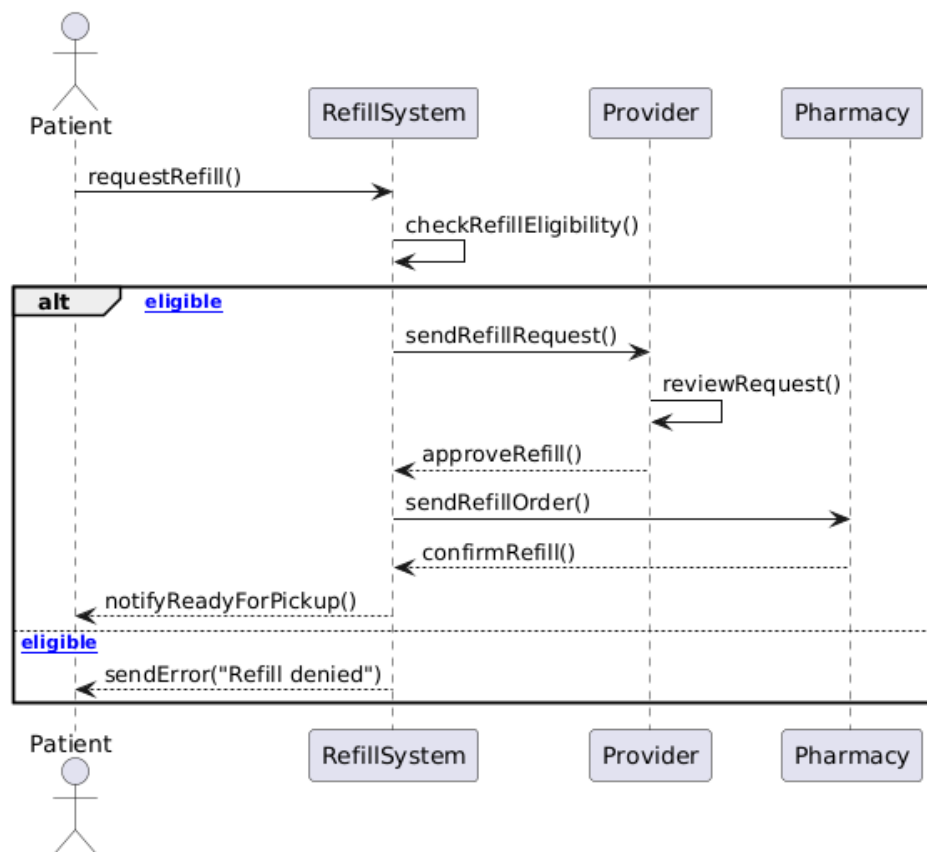
Actor: Patient

Object: RefillSystem, Provider, Pharmacy

Steps of this interaction:

1. The patient requests a refill for their prescription.
2. The RefillSystem will check whether the patient is due for a prescription refill.
3. If the patient is due for a refill, the RefillSystem will send the refill request to the Provider. If the prescription can't be refilled, the RefillSystem will send an error message back to the patient and deny the request.
4. The Provider will see the request and approve the refill.

5. The RefillSystem will send the request for a refill to the Pharmacy.
6. The Pharmacy will accept the request and refill the prescription.
7. The RefillSystem will notify the patient that the refill request has been sent to the Pharmacy and can be picked up there.



Activity Diagrams

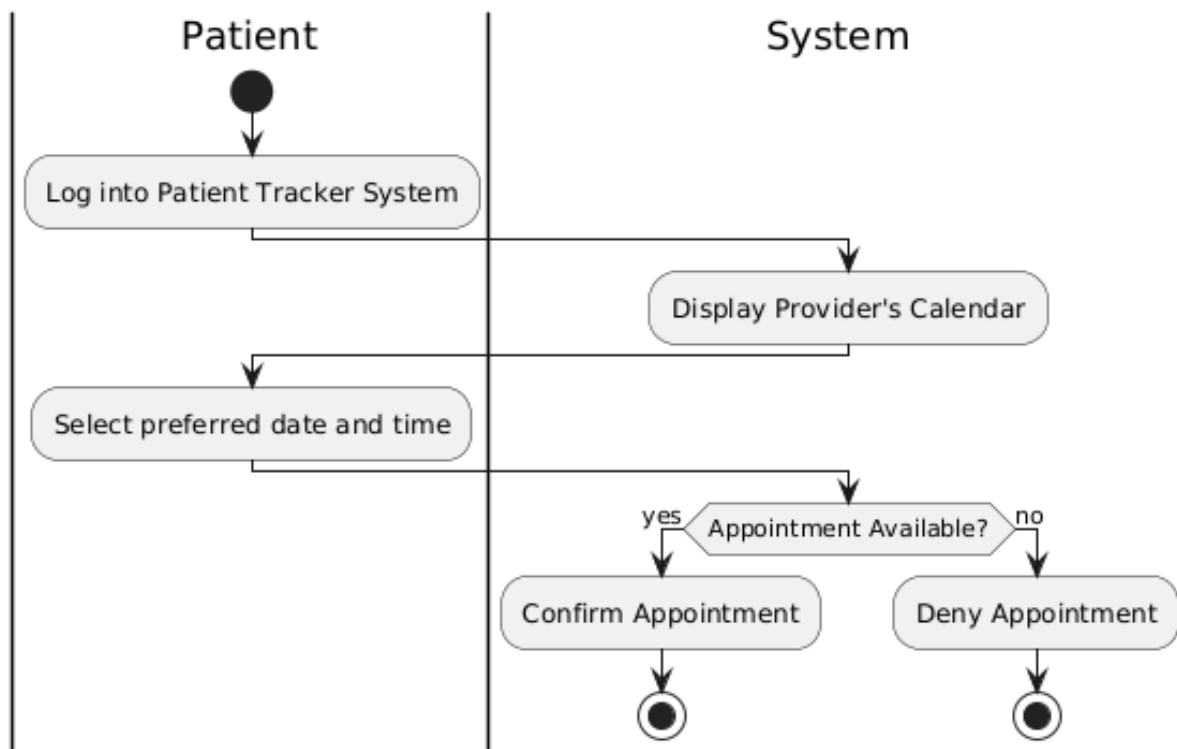
Schedule an Appointment:

States:

- Initial State: The patient logs onto the patient tracker system.
- Final State:
 1. The appointment is accepted.
 2. The appointment is denied because it isn't available.

Actions:

The patient pulls up the calendar to view the provider's availability. The system brings up the provider's calendar, and the patient can choose a day and time that works for them.



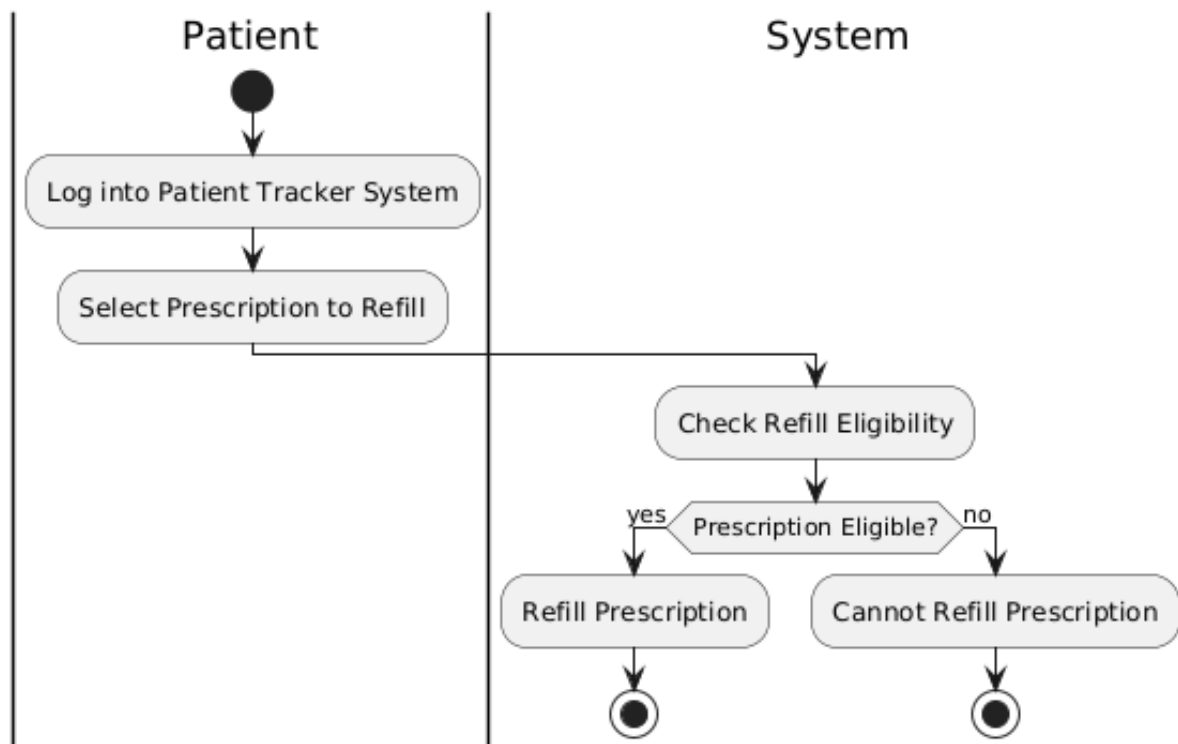
Request a Refill for a Prescription:

States

- Initial State: The patient logs onto the patient tracker system.
- Final State:
 1. The prescription is refilled.
 2. The prescription is unable to be refilled.

Actions:

The patient selects the prescription they would like refilled. The system then checks whether the prescription is eligible to be refilled or not.



Traceability Matrix for Patient Tracker System

System Requirements

No.	Priority Weight (1–5)	Description
REQ1	5	The system shall allow patients to create an account and update their personal information.
REQ2	4	The system shall allow patients to view their full medical records and treatment plans.
REQ3	4	The system shall allow patients to schedule, reschedule, and cancel appointments.
REQ4	4	The system shall allow patients to request prescription refills.
REQ5	3	The system shall allow patients to view their test results.
REQ6	5	The system shall support secure communication between patients and providers.
REQ7	3	The system shall allow providers to view their schedules and patient lists.
REQ8	4	The system shall allow providers to update treatment plans and manage prescriptions.
REQ9	4	The system shall allow administrators to manage and schedule appointments, including assigning providers.

Use Cases

No.	Description	Priority (1–5)
UC1	Patient creates an account on their own.	2

REQ6 (5)							x					x			5	10
REQ7 (3)								x	x						3	6
REQ8 (4)										x	x				4	8
REQ9 (4)				x									x	x	4	12

Explanation

This traceability matrix shows how each system requirement connects to one or more use cases.

High-priority requirements (PW 4–5) such as account creation, communication, scheduling, and prescriptions are addressed in early use cases (UC1, UC4, UC5, UC7, UC12, UC13, UC14).

Medium-priority requirements (PW 2–3) enhance usability, such as test result viewing or schedule access. This prioritization supports an incremental implementation plan focusing first on core patient and provider functionality.

System Architecture and System Design

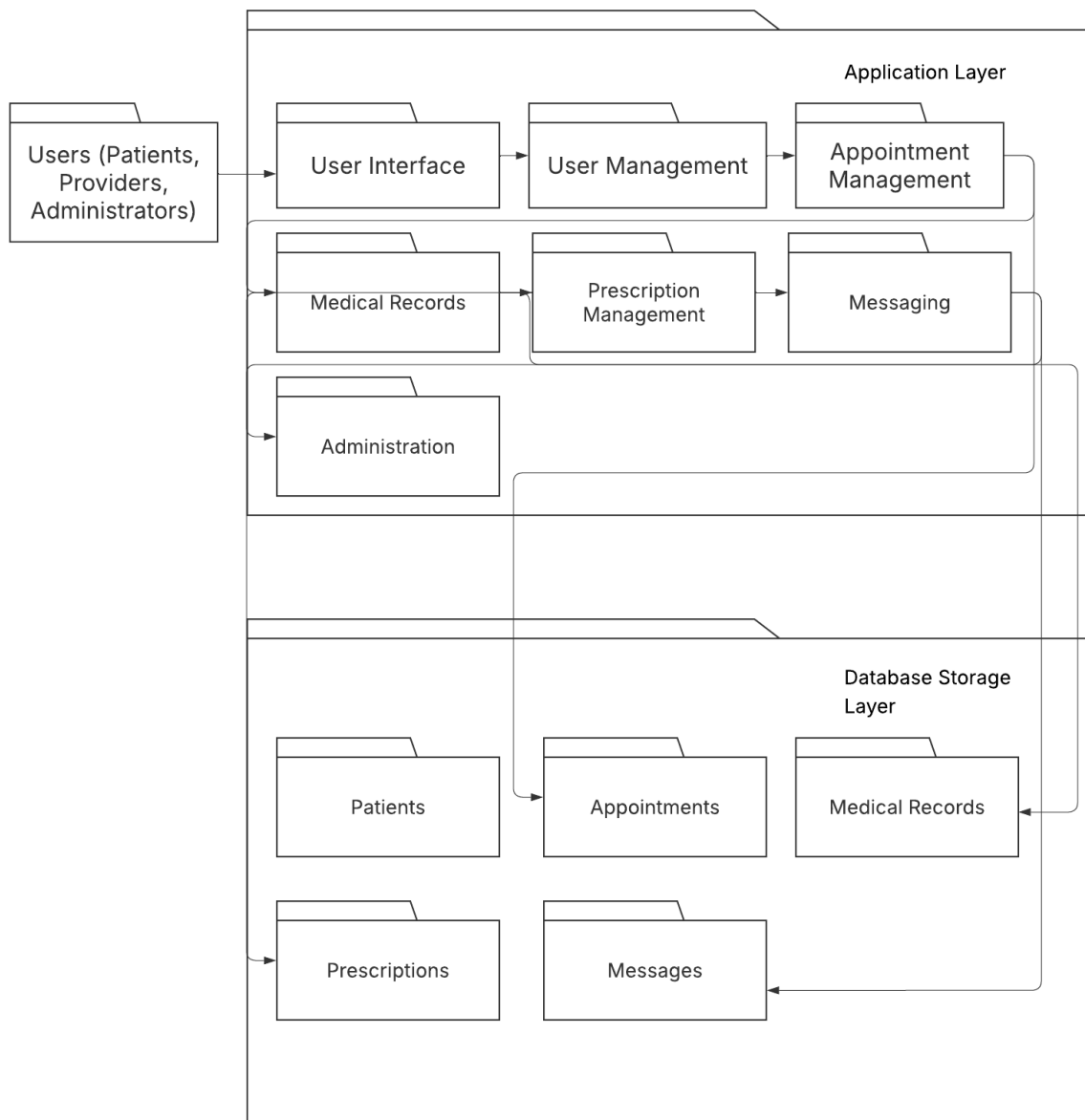
Architectural Styles

My Patient Tracker System uses a client-server architectural style, meaning the user (patient, provider, administrator) communicates with a server. The client does not know what the server does with the information they send. The client side waits for the results that the server produces. I use MAMP to run the Patient Tracker System. The server produces the web pages. Users' requests are fulfilled by PHP. Data, such as patient information, appointment information, prescriptions, and patient medical records, is collected and stored in the MySQL database. The user interface is run by the client side, and all the data from the database is stored on the server. A local server on my computer runs the Patient Tracker System.

Identifying Subsystems

The UML package diagram I have below shows what subsystems the Patient Tracker System consists of. The client side consists of an application layer, and the server has a database storage layer. The application layer has a User Interface package that uses HTML, CSS, and JavaScript. The system also has a User Management package, which allows users to create an account and log in. There is also an appointments package that allows users to create and manage appointments. The Medical Records package shows patients a full scope of their medical history. A Prescription Management package is used to help patients request refills on prescriptions from their providers. The **Communication package** allows patients and providers to message one another. The last package is the **Administration package, which allows the admin to view patient information, manage appointments, etc.** The packages can send and collect information from the MySQL database.

UML Package Diagram



Persistent Data Storage

The Patient Tracker System must store and save patient information, appointments, prescriptions, and medical records even after it is closed. MySQL is the database I am using.

Global Control Flow

- **Executive Orders**

The Patient Tracker System waits for users to interact with it and then handles the information. The user does not have to do actions in any order. The system knows how to handle all the things it can do, regardless of when or how the user asks for it.

- **Time Dependency**

Actions do not have to follow a specific timeline. The system is designed to respond to any user request. New data is automatically stored in the database. Information that is updated is automatically updated in the database.

- **Hardware Requirements**

- Display: Color screen, minimum resolution 640×480 pixels
- Computer: Desktop or laptop (Mac/PC)
- Memory: 1 GB RAM
- Hard Drive: 1 GB free space
- Software: MAMP (Apache, MySQL, and PHP)

Algorithms and Data Structures for Patient Tracker System

Algorithms

The Patient Tracker System uses the following algorithms:

- **Login Algorithm-** The system checks the database to ensure that when a user logs in, they use a username/password that has been stored previously. If the username and password match those in the database, the user is allowed into the system and redirected to the patient dashboard.
- **Appointment Sorting Algorithm-** The system displays appointments in chronological order, so upcoming appointments are closest to the top of the list, and future appointments are underneath. The system does this with sorting.
- **Registration Algorithm-** When someone registers for an account, the system ensures that all fields are filled and that the username/email/password are not in the database yet. If they are, this indicates that a user account already exists. If it doesn't, the user's information is saved in the database, and a new account is registered.
- **Medical Record Updating Algorithm-** When a user adds to or makes changes to their medical record, the system must get their record and update the proposed changes. The changes are then able to be seen by the user.

Data Structures

The Patient Tracker System uses the following data structures:

- **Arrays-** These store things like appointments, patient information, and medical records temporarily, only while the system is running. I chose to use arrays because they make it easy to access the data quickly.

- Database Tables – These store more permanent information about patients, providers, appointments, and medical records. The tables use foreign keys, which help keep the data organized.

User Interface Design and Implementation, Design of Tests for Online Course Management System

System Overview

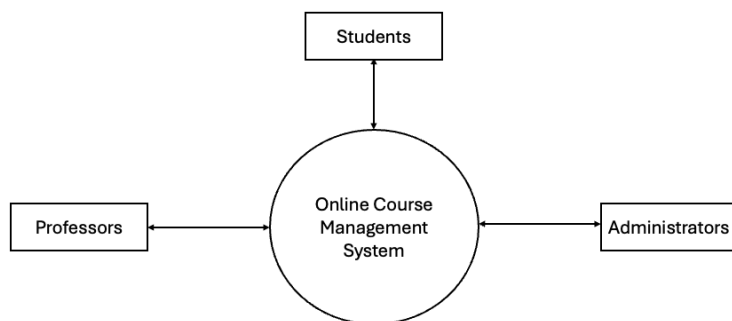
The Online Course Management System is a system that a university uses to allow students to complete courses and interact with peers and professors completely online. Students can submit assignments and keep track of their grades through the software. Professors use the system to manage their courses and work with students. Administrators of the Online Course Management System can add new users (students or professors) to the system.

High-Level System Goals and Objectives

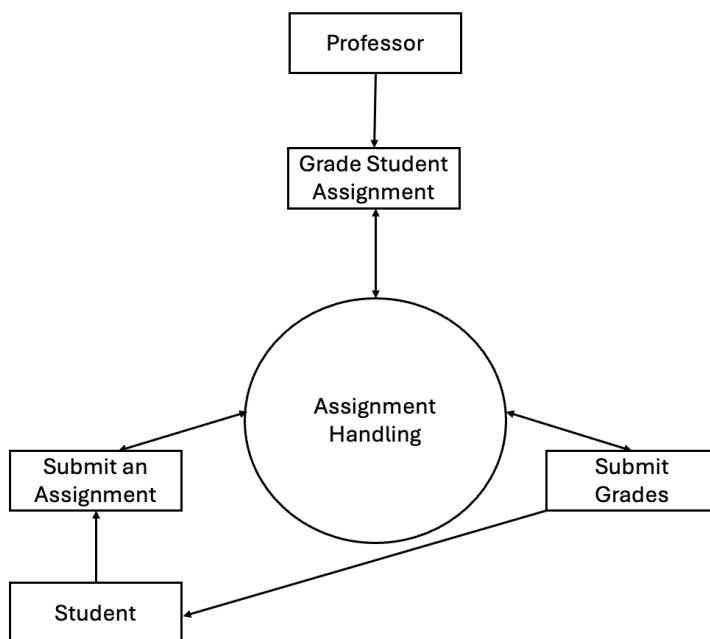
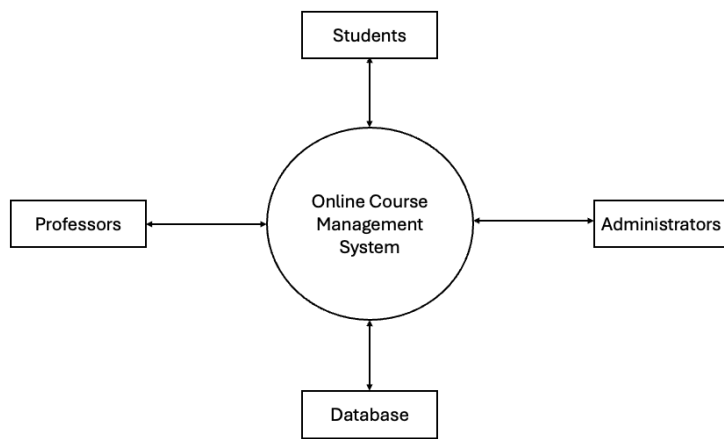
- Make it easier for students to navigate online courses.
- Ensure messaging abilities between students and professors.
- Make it easier for professors to manage their classes.
- Make it easier for professors to grade student assignments.

Design Integration

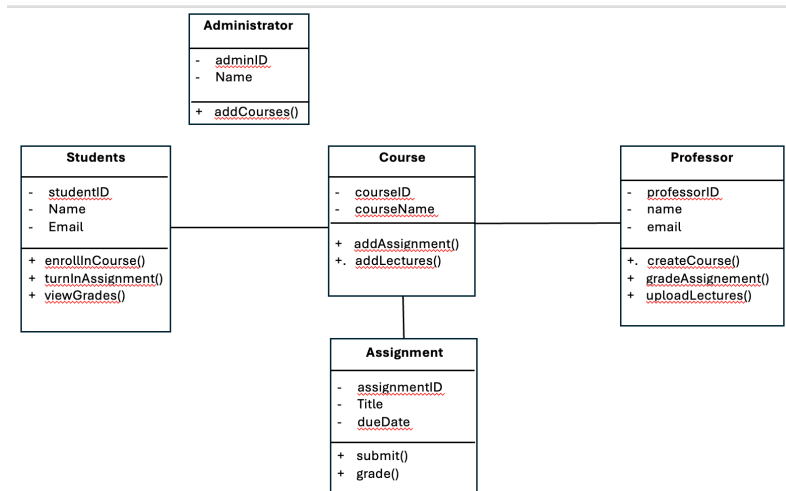
Context Diagram



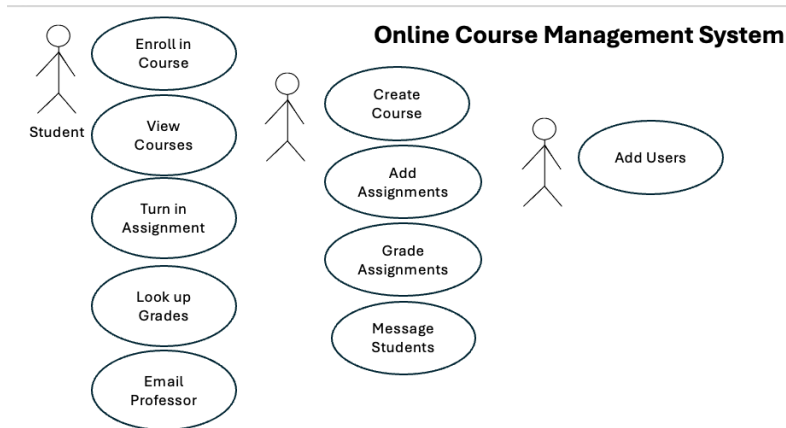
Data Flow Diagrams



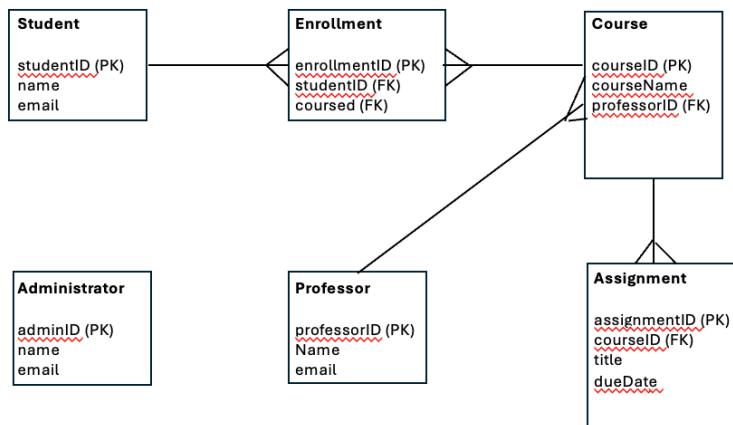
Object Model with Relationships



Use Case Diagram



Entity-Relationship Diagram



Architecture and UI

System Architecture Diagram

Browser → Server → Database

UI Mockups

Student Dashboard:

Welcome, Student!

Upcoming Assignments:

- System Design Final Project
- Final Analysis of Health Trends

My Courses:

- System Design
 - Quantitative Analysis of Health Information
-

Assignment Submission Page:

System Design Final Project

12/15/2025

100 Points Possible

Upload:

Create New Message Page:

To:

John Doe

Subject:

Assignment Help

Message:

Navigation Flow and Usability Considerations

Front-End: HTML/CSS/JavaScript

Backend: Web browser

Database: MySQL

Security and Risk Consideration

Security Features

- Different accessibility depending on whether you are a student, professor, or administrator
- Username/password required to gain access into the system.

- Need permission to edit courses, grade assignments, etc.
- File scanning to make sure the assignments being uploaded do not carry viruses.

Potential Design-Level Risks and Mitigation Strategies

- Potential for Data loss- Make sure the database is regularly being backed up.
- Plagiarism by Students- Enlist the help of a plagiarism checker to ensure student work is their own.
- High traffic around deadlines- Use methods like caching to make it easier to pull up data that is used often.

Design Patterns and Best Practices

- Model-View-Controller

The MVC pattern separates the system into three main parts: the Model, the View, and the Controller. The Model handles student information and assignments. The View part is what the user sees, like the student and professor dashboard, assignment submission pages, courses page, etc. The Controller takes what the user does and can update the system as needed.

Repository Pattern

The Repository Pattern is a way to manage the database by putting all the code for the database in one place. This makes it easier to keep the system consistent and easier to update.

Project Planning and Development Approach:

Software

- **Frontend:** React.js • **Backend:** Python
- **Database:** MySQL

Hardware

- Computers
- A server

Network

- Quick internet access will be ideal, so patients can get the most up-to-date information about their health quickly.
- The network will be protected by firewalls/encryption, so that patient health information is not breached or shared with people who should not have access.

Development Plan:

- **Weeks 1–2:** The database will be set up, and the backend will be connected to the frontend.
- **Weeks 3–4:** User registration will be implemented, and login for different users (patients, doctors, admins, etc.) will be tested.
- **Weeks 5–6:** Patient account management will be set up so they can update information as it changes.

- **Weeks 7–8:** The appointment scheduling and the reminder system will be created.
- **Weeks 9–10:** Administrator management capabilities will be added, allowing them to access patient and staff accounts.
- **Weeks 11–12:** The system will be tested, and any errors will be fixed.
- **Weeks 13–14:** Usability and user experience will be improved.
- **Week 15:** Final testing will be performed, and the system will be prepared for release.